LAPORAN CODE AUDIT - CLEAN CODE

Nama: Rohmi Ihsan
Mata Kuliah: Rekayasa perangkat lunak
Tugas: Misi Minggu 5 - Code Audit
Aplikasi: Sistem Checkout & Pembayaran (Python)

Link GitHub: https://github.com/rohmiihsan67-hash/Rekayasa-perangkat-lunak

Pelanggaran Clean Code:

1. SRP dilanggar: Satu class menangani banyak tugas.
2. OCP dilanggar: Penambahan metode pembayaran harus mengubah kode.
3. DIP dilanggar: Tidak menggunakan abstraction.
4. Hardcoded logic pada metode pembayaran.
5. Tidak modular dan sulit dikembangkan.

Aspek	Sebelum	Sesudah
Struktur	1 class besar	Dipisah menjadi beberapa class
SRP	Tidak diterapkan	Sudah diterapkan
OCP	Harus ubah kode	Bisa tambah tanpa ubah kode
DIP	Tidak ada	Menggunakan abstraction
Maintainability	Sulit	Mudah dikembangkan

Prinsip Clean Code yang Diterapkan:

• SRP: Setiap class memiliki satu tanggung jawab.
• OCP: Sistem dapat dikembangkan tanpa mengubah kode lama.
• DIP: Bergantung pada abstraction, bukan implementasi.
• Modularity: Kode dipisah menjadi bagian kecil.
• Readability: Mudah dibaca dan dipahami.

Penjelasan Kode Sebelum dan Sesudah Perbaikan

1. Analisis Kode Sebelum Perbaikan (Bad Code)

Pada implementasi awal, sistem checkout dikelola oleh satu kelas utama yaitu:
class OrderManager
Kelas ini menangani seluruh proses mulai dari pembayaran, pengiriman notifikasi, hingga pembaruan status pesanan. Berdasarkan hasil analisis, terdapat beberapa pelanggaran terhadap prinsip *Clean Code*, di antaranya: ***a. Pelanggaran Single Responsibility Principle (SRP)***

Kelas OrderManager memiliki lebih dari satu tanggung jawab, yaitu:

- Memproses pembayaran
- Mengirim notifikasi kepada pengguna
- Mengubah status pesanan

Hal ini menyebabkan kode menjadi sulit untuk dipelihara dan dikembangkan, karena setiap perubahan pada satu fungsi dapat berdampak pada fungsi lainnya.

b. Pelanggaran Open/Closed Principle (OCP)

Penggunaan struktur kontrol seperti: if
payment_method == "credit_card":
menunjukkan bahwa setiap penambahan metode pembayaran baru mengharuskan perubahan pada kode yang sudah ada. Hal ini bertentangan dengan prinsip OCP, di mana sistem seharusnya terbuka untuk pengembangan namun tertutup untuk modifikasi.

c. Pelanggaran Dependency Inversion Principle (DIP)

Kelas OrderManager secara langsung bergantung pada implementasi konkret (hardcoded logic), tanpa menggunakan abstraksi. Hal ini mengakibatkan ketergantungan yang tinggi (*tight coupling*) dan mengurangi fleksibilitas sistem.

d. Hardcoded Logic

Logika pemrosesan pembayaran ditulis langsung di dalam metode, seperti:
print("Processing Credit Card...")
Pendekatan ini membuat kode sulit untuk diperluas dan diuji.

e. Kurangnya Modularitas

Seluruh logika ditempatkan dalam satu kelas dan satu metode besar, sehingga:

- Sulit dibaca
- Sulit diuji
- Sulit dikembangkan lebih lanjut

2. Analisis Kode Setelah Perbaikan (Clean Code)

Setelah dilakukan refactoring, struktur kode diubah menjadi lebih modular dengan menerapkan prinsip-prinsip *Clean Code*.

a. Penerapan Single Responsibility Principle (SRP)

Tanggung jawab sistem dipisahkan ke dalam beberapa kelas, antara lain:

- CreditCardProcessor, QrisProcessor → menangani pembayaran
- EmailNotifier → menangani notifikasi

- CheckoutService → mengatur alur proses checkout

Dengan pemisahan ini, setiap kelas hanya memiliki satu tanggung jawab yang jelas.

b. Penerapan Open/Closed Principle (OCP)

Penambahan metode pembayaran baru dilakukan dengan membuat kelas baru yang mengimplementasikan interface IPaymentProcessor, tanpa perlu mengubah kode yang sudah ada.

Contoh:

class QrisProcessor(IPaymentProcessor):

Hal ini membuat sistem lebih fleksibel dan mudah dikembangkan.

c. Penerapan Dependency Inversion Principle (DIP)

Kelas CheckoutService tidak lagi bergantung pada implementasi konkret, melainkan pada abstraksi:

def __init__(self, payment_processor: IPaymentProcessor, notifier: INotificationService):

Pendekatan ini mengurangi ketergantungan antar komponen dan meningkatkan fleksibilitas sistem.

d. Peningkatan Modularitas

Kode dibagi menjadi beberapa komponen kecil yang saling terpisah, sehingga:

- Lebih mudah dibaca
- Lebih mudah diuji
- Lebih mudah dipelihara

e. Peningkatan Readability dan Maintainability

Dengan penggunaan nama kelas dan metode yang lebih deskriptif serta struktur yang lebih terorganisir, kode menjadi lebih mudah dipahami oleh pengembang lain.

3. Perbandingan Sebelum dan Sesudah Perbaikan

Aspek	**Sebelum Perbaikan**	**Sesudah Perbaikan**
Struktur	Terpusat pada satu kelas	Terpisah menjadi beberapa kelas
SRP	Tidak diterapkan	Diterapkan dengan baik
OCP	Harus memodifikasi kode lama	Dapat menambah fitur tanpa modifikasi
DIP	Bergantung pada implementasi langsung	Menggunakan abstraksi
Maintainability	Rendah	Tinggi
Scalability	Sulit dikembangkan	Mudah dikembangkan

4. Kesimpulan

Berdasarkan hasil analisis, kode awal memiliki beberapa pelanggaran terhadap prinsip *Clean Code*, khususnya SRP, OCP, dan DIP. Setelah dilakukan refactoring, struktur kode menjadi lebih modular, fleksibel, dan mudah dikembangkan.

Penerapan prinsip *Clean Code* tidak hanya meningkatkan kualitas kode dari sisi teknis, tetapi juga mempermudah proses pemeliharaan dan pengembangan sistem di masa mendatang